ML-based Deep Bitstream Inspection for Serverless FPGAs

SANTIAGO DE HEREDIA, Systems Group, ETH Zurich, Switzerland
MAXIMILIAN J. HEER, Systems Group, ETH Zurich, Switzerland
GUSTAVO ALONSO, Systems Group, ETH Zurich, Switzerland

Cloud and serverless FPGA platforms expose reconfigurable fabric to user-submitted designs, creating risks from malicious circuits such as ring oscillators. Existing machine-learning approaches typically inspect bitstreams offline and require additional hardware, including CPUs and GPUs. In moving towards an accelerator model where the FPGA functions as a standalone unit, we instead study an on-FPGA checker that analyzes incoming partial bitstreams before reconfiguration. We generate benign accelerator applications and malicious bitstreams, implement an end-to-end machine learning preprocessing and inference pipeline, and synthesize the model directly into the FPGA using hls4ml. Our selected quantized model achieves 88.5% F1 and 95.1% ROC AUC. We overlap preprocessing and CNN execution through streaming and reduce resource use through hls4ml FIFO sizing and arithmetic reuse. On an Alveo U55C, median local-inspection latency is 2.679–3.074 ms across eight real bitstreams of 5.28–53.10 MiB. Measurements start with the bitstream available in FPGA memory and end when the verdict is available inside the FPGA, excluding input preload and reconfiguration. Offloading the same inputs over PCIe peer-to-peer to an AMD MI210 GPU running the compiled float model takes 1.327–8.012 ms. Local inspection is faster for six of the eight inputs, reaching a 2.91× speedup at 53.10 MiB; GPU offloading is faster for the two smallest inputs. The synthesized checker uses 4.95% of device LUTs, 1.44% of BRAM, 7.36% of DSPs, and 3.54% of URAM, excluding the Coyote shell. These results demonstrate the feasibility of an on-FPGA checker for detecting malicious bitstreams before partial reconfiguration.

CCS Concepts: • **Security and privacy** → **Systems security**; • **Computer systems organization** → *Cloud computing*; • **Computing methodologies** → *Machine learning*; Neural networks.

Additional Key Words and Phrases: cloud FPGA security, bitstream inspection, partial bitsreams, malicious circuit detection, serverless FPGAs, partial reconfiguration, convolutional neural networks, ML for systems, hls4ml

ACM Reference Format:
Santiago de Heredia, Maximilian J. Heer, and Gustavo Alonso. 2026. ML-based Deep Bitstream Inspection for Serverless FPGAs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Field-programmable gate arrays (FPGAs) are increasingly used as datacenter accelerators because they combine programmability with high performance and energy efficiency [3, 5]. Cloud and serverless FPGA platforms extend this model by allowing users to deploy hardware applications on shared infrastructure. Partial reconfiguration (PR) makes this practical by loading user logic into a reconfigurable region while the trusted FPGA shell remains active [13, 20].

Authors' Contact Information: Santiago de Heredia, sdeheredia@ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland; Maximilian J. Heer, maximilian.heer@inf.ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland; Gustavo Alonso, alonso@inf.ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

This deployment model exposes shared FPGA resources to untrusted user logic. Malicious circuits such as ring-oscillator (RO) arrays can generate sustained switching activity, causing voltage drops, timing faults, and denial of service [4, 10, 14, 19]. Demonstrated attacks on cloud FPGA infrastructure therefore motivate inspecting user bitstreams before partial reconfiguration [14].

Prior work has shown that machine-learning models can identify malicious designs from bitstream-derived representations without fully reconstructing their circuit semantics [6, 9, 18, 21]. However, these approaches are commonly evaluated as software-based and offline classification pipelines, including the need for additional hardware asides from the FPGA, especially GPUs for inference[7]. A serverless FPGA platform introduces additional requirements and goals that make these solutions undesired:

- **Latency:** Bitstream inspection should not contribute significantly to the latency of deploying said bitstream.
- **Economics and Hardware:** Ideally, the accelerator system should be kept simple and inexpensive, avoiding the need for additional servers and / or specialized hardware.

Offline approaches trade complexity at the expense of blocking deployment of the bitstream until inspection has taken place in a separate process. Moreover, reserving additional specialized hardware for (online) bitstream inspection and inference (especially with GPUs) make this option less attractive for commercial ventures like AWS F2**[TODO]**.

We investigate an *on-FPGA checker* placed before partial reconfiguration of the user bitstream. The checker preprocesses the raw bytes of the incoming bitstream, executes a quantized convolutional neural network (CNN), and returns a suspiciousness score to the deployment logic. We study this design in the context of Coyote, a datacenter-grade FPGA shell supporting PR and multitenancy [20]. We generate benign Coyote application bitstreams and RO-based malicious bitstreams, train compact CNNs, and synthesize and optimize selected models using hls4ml [1, 8].

The resulting checker provides useful detection at millisecond-scale latency while leaving sufficient resources for the Coyote shell and user logic.

This work makes the following four contributions:

- We construct a dataset of benign Coyote partial bitstreams and RO-based malicious bitstreams across multiple floorplans and payload footprints.
- We develop an on-FPGA inspection pipeline combining deterministic byte-stream preprocessing, quantized CNN inference, and hls4ml synthesis.
- We evaluate detection quality, latency, and routed FPGA resource overhead together, identifying a practical deployment point for pre-reconfiguration inspection.
- We compare our on-FPGA checker system against an GPU checker integrated with the FPGA, demonstrating lower-or-parity end-to-end latency compared to the baseline.

2 Background, Threat Model, and Related Work

2.1 Deployment Setting and Threat Model

Serverless FPGA platforms allow users to deploy hardware accelerators on shared reconfigurable infrastructure. Partial reconfiguration (PR) supports this model by dividing the FPGA into a trusted static shell and one or more reconfigurable regions containing user logic. The shell remains active while individual user regions are reconfigured, allowing the platform to retain control over host communication, memory, networking, and runtime services [13, 20].

This separation between a static shell and a reconfigurable region creates a security boundary. A user-submitted partial bitstream is an untrusted input that directly determines the circuit instantiated inside a reconfigurable region. Malicious circuits can exploit physical resources shared across the device, including power delivery, thermal headroom, and timing margins. We focus on

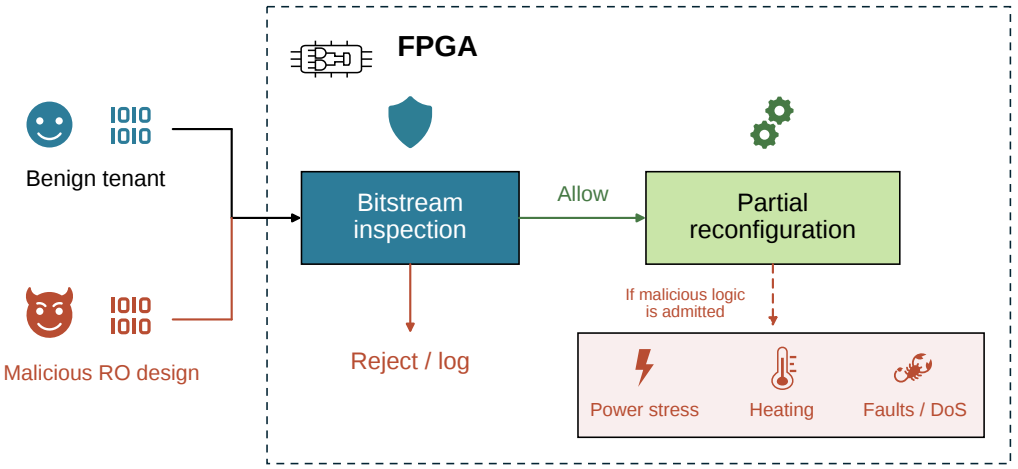


Fig. 1. Threat model and proposed inspection gate for a shared FPGA. Tenant partial bitstreams are inspected before partial reconfiguration. Rejected submissions are blocked and logged. The dashed arrow shows how an admitted malicious circuit can affect other tenants through shared physical resources. The complete admission path is proposed; this work implements and evaluates the checker.

ring-oscillator (RO) payloads, which produce sustained switching activity and can induce voltage drops, timing faults, or denial of service when instantiated at sufficient scale [4, 10, 14, 19].

We consider an attacker who can submit a syntactically valid partial bitstream for a user region. The bitstream contains an RO-based payload intended to pass existing platform checks and stress shared physical resources after configuration.

The defender places an on-FPGA checker before PR. The checker inspects the bitstream that would subsequently be provided to the PR controller. It produces a maliciousness score that the trusted deployment logic converts into an allow or reject decision. A true positive blocks and logs a suspicious bitstream. A false negative would allow a malicious bitstream to be configured, while a false positive would reject a benign accelerator applicator.

Figure 1 summarizes the threat model and the proposed placement of inspection before reconfiguration.

2.2 Existing Defenses and Research Gap

Existing defenses against malicious FPGA designs broadly fall into three groups. Table 1 summarizes their relationship to this work.

Platform and vendor checks are already part of practical cloud-FPGA deployment flows, but attacks have shown that syntactically valid power-wasting circuits can pass such checks [14]. Structural approaches instead search for specific circuit patterns, including self-oscillators [15, 24]. These methods provide direct evidence of suspicious structures, but often depend on device-specific architectural knowledge or bitstream reverse engineering.

Machine-learning approaches avoid explicit reconstruction of the implemented circuit. Prior work has classified malicious designs using features derived from FPGA bitstreams, including artificial neural networks, CNNs, and other supervised classifiers [6, 9, 18, 21]. These studies establish

Table 1. Comparison of malicious-bitstream detection approaches across deployment and implementation dimensions.

Approach	Detection basis	Deployment		Properties		Target
		Deployment integrated	On-FPGA	No reverse engineering	Datacenter target	
Platform checks	Provider-defined rules	✓	×	✓	✓	Cloud deployment
Structural scanning	Circuit structures	×	×	×	×	Device-specific FPGA
Offline ML	Learned features	×	×	✓	×	Host software
Embedded ML	Byte-frequency features	×	×	✓	×	PYNQ-Z1
This work	**Learned byte patterns**	**Proposed**	✓	✓	✓	**Coyote/U55C**

Legend: ✓indicates that a property is demonstrated, while ×indicates that it is not demonstrated. "Proposed" denotes integration designed as part of this work but not yet connected to the complete Coyote control path.

that bitstream-level representations contain learnable information. However, they primarily evaluate detection as an offline software task and do not jointly consider placement in a datacenter FPGA deployment path, FPGA resource cost, and inspection latency.

Embedded approaches demonstrate that bitstream inspection can execute locally on a PYNQ-Z1 platform without external computation [23]. BLADEI further investigates deployment latency and proposes hardware-accelerated preprocessing, while retaining classification on the embedded ARM processor [22]. Serverless FPGA deployment requires balancing inspection latency with hardware resource usage. Screening delays contribute directly to deployment time, while checker resources compete with the platform shell and user accelerators. Our work places both preprocessing and quantized model inference in programmable logic with hls4ml, and optimizes their execution through streaming and resource tuning. We evaluate the resulting checker on Coyote and the Alveo U55C, measuring detection quality, inspection latency, and FPGA resource cost together.

This leaves three feasibility questions:

(1) **Can the checker retain useful detection quality, particularly a sufficiently low false-negative rate?**
(2) **Can the checker fit alongside the Coyote shell and user logic while adding acceptable inspection latency?**
(3) **Can preprocessing, inference, and decision logic be placed on the FPGA and deployment path, without a separate host-side classification stage?**

The following sections evaluate these questions through dataset construction, model training, hardware synthesis, and a proposed Coyote integration.

3 Dataset and Preprocessing

3.1 Dataset

We construct a binary classification dataset of benign Coyote partial bitstreams and malicious RO-based partial bitstreams generated through the Coyote partial-reconfiguration flow. The benign

Table 2. Seven-stage quantized CNN architecture used by the on-FPGA checker. All convolutional stages use quantized weights and ReLU activations.

Stage	Filters	Kernel	Stride	Pooling
Conv 0	8	5×5	2×2	2×2
Conv 1	16	3×3	1×1	2×2
Conv 2	24	3×3	1×1	2×2
Conv 3	24	3×3	1×1	2×2
Conv 4	32	3×3	1×1	2×2
Conv 5	32	3×3	1×1	2×2
Conv 6	32	3×3	1×1	2×2
Output	1	–	–	AvgPool

class contains eight Coyote applications spanning HLS, cryptographic, communication, and compute examples. We build these applications across multiple Coyote floorplans, yielding 221 benign partial bitstreams.

Malicious samples use the RO-based designs of Mahmoud et al. [16], instantiated at different payload sizes and floorplans. The corpus contains 225 malicious partial bitstreams.

3.2 Preprocessing

We uniformly downsample each raw partial bitstream to $H \cdot W$ bytes, using evenly spaced positions from the first byte to the last and rounding each position down to an integer index. Shorter bitstreams are padded with zero bytes to reach the same length. We then invert and normalize each byte value b as $(255 - b)/255$ and reshape the sequence into an $H \times W \times 1$ grayscale tensor. Padding therefore produces pixels with value 1.

Our final checker uses a 256×256 input, corresponding to 65,536 sampled bytes per partial bitstream. We evaluate additional input resolutions during the model design-space exploration in Section 4.

4 ML-Based Bitstream Inspection

We use convolutional neural networks (CNNs) to classify the fixed-size bitstream representations introduced in Section 3. Prior work has shown that bitstream-derived representations contain learnable structure and can be used to distinguish benign and malicious FPGA designs [6, 9, 18, 21]. CNNs are also well suited to our deployment goal because quantized models can be translated into FPGA circuits using tools such as hls4ml [1, 8].

4.1 CNN Architecture

The selected checker uses a seven-stage convolutional architecture operating on a 256×256 byte-image tensor. Each stage consists of a quantized convolution, quantized ReLU activation, and max pooling. Channel width increases from 8 filters in the first stage to 32 filters in the later stages. A final average-pooling operation reduces the remaining feature map before a single quantized dense output produces the classification logit.

Table 2 summarizes the architecture. We use quantized weights and activations and do not use batch normalization. The output logit is converted to a maliciousness probability using a sigmoid; the decision threshold is applied later by the deployment logic.

4.2 Training

Models are implemented in TensorFlow [2] using QKeras [11]. We use quantization-aware training [11], so that the model is trained under the same fixed-precision behavior used during hardware generation. Training minimizes binary cross-entropy from logits. We augment the training data with horizontal and vertical flips and use a balanced production training set containing equal numbers of benign and malicious samples.

During model development, we use stratified cross-validation to compare model configurations [12]. After selecting the final architecture, we train a production model separately and evaluate it on the held-out test set described in Section 6.1.

4.3 Hardware-Aware Design-Space Exploration

Model selection considers hardware feasibility in addition to software classification quality. Rather than exhaustively evaluating every combination of parameters, we explore the design space in stages. We first vary input resolution and CNN depth to determine how model capacity and retained bitstream information affect classification quality. We then evaluate quantization and pruning as model-compression techniques before passing selected networks to the hardware synthesis flow described in Section 5.

The principal model-level dimensions are therefore input resolution, network depth, and numerical precision. Hardware parallelism is explored separately after conversion to hls4ml by varying operation reuse and per-layer configuration.

Figure 2 summarizes the software exploration. Detection quality generally improves with input resolution and CNN depth, with smaller gains at the largest configurations. These are pooled-validation results used for model selection, distinct from the held-out test results in Section 6.2. Considering detection quality and hardware cost, we select the 256×256, seven-stage quantized CNN. This is the model synthesized and evaluated in the remainder of the paper.

5 On-FPGA Checker

We next map the trained bitstream classifier into FPGA logic and consider how the resulting checker can be placed in the Coyote deployment path. The hardware design consists of two components: deterministic byte-stream preprocessing and the quantized CNN classifier. Importantly, both execute on the FPGA, allowing inspection to take place without a separate host-side classification stage. Figure 3 shows how these components could support a standalone, network-attached Coyote FPGA.

5.1 Hardware Generation

We convert the trained QKeras CNN to hardware using hls4ml [1, 8]. hls4ml translates the quantized network into an HLS implementation, which is first evaluated through C simulation and subsequently synthesized to RTL. The final models use quantization-aware training, so the weight and activation precision used during training already reflects the quantized representation targeted by the generated hardware.

Before synthesis, we compare the hls4ml C-simulation output against the corresponding QKeras model. We require matching classifications on the validation inputs and inspect output deviations before synthesis. This provides a parity check between the trained model and its fixed-point hardware implementation.

The preprocessing operation from Section 3.2 is implemented separately in HLS C++ and lowered to RTL. It operates directly on the incoming bitstream bytes and reproduces the deterministic sampling, padding, inversion, and normalization used during training. The board latency reported

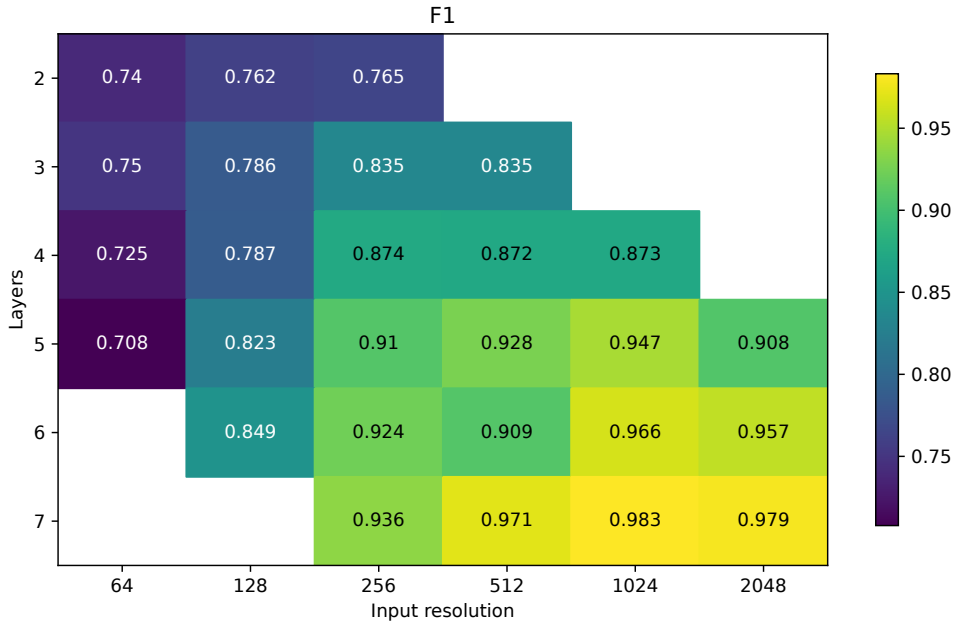


Fig. 2. Pooled-validation F1 across input resolution and CNN depth. This software exploration informs selection of the 256 × 256, seven-stage checker.

in Section 6.3 measures host invocation to verdict availability inside the FPGA, including input transport and overlapping preprocessing and inference.

5.2 Hardware Optimization

Once the trained network is fixed, hls4ml exposes a hardware design space that trades resource usage against inference latency. The main parameter is the *reuse factor*, which controls how aggressively arithmetic units are shared across operations. Lower reuse increases parallelism and reduces latency at the cost of additional hardware, while higher reuse reduces resource use but increases execution time.

We initially evaluate resource-oriented hls4ml configurations and sweep reuse factors for the selected CNN architectures. A single global reuse setting, however, provides only a coarse trade-off because convolutional stages differ substantially in their operation counts and hardware cost. We therefore also apply manual per-layer reuse settings, assigning greater parallelism to latency-dominant layers while retaining more sharing elsewhere.

Quantization and pruning are additionally explored as hardware-aware model transformations. We train the quantized checker through QKeras quantization-aware training. For each transformed model, we retain a configuration only when software detection quality and hls4ml prediction parity remain acceptable. With the selected weights fixed, we then overlap preprocessing with inference and tune FIFO depths and per-layer reuse through hls4ml configuration. Section 6.3 evaluates the resulting latency and routed FPGA resources.

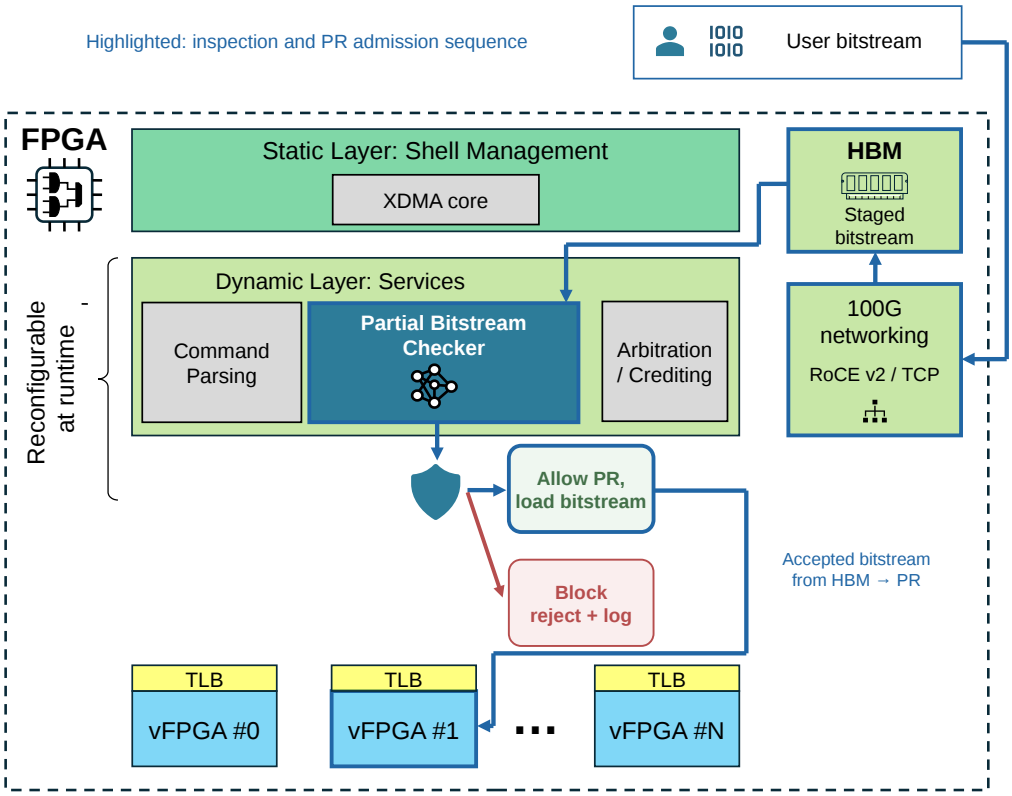


Fig. 3. Proposed standalone Coyote inspection accelerator. The highlighted sequence follows a user bitstream through 100G network ingress, HBM staging, inspection, and admission to PR of a selected vFPGA. The checker produces a verdict; PR loads the accepted raw bitstream from HBM after complete input reception. The original Coyote services and user regions are retained. Submission uses networking rather than the shell's XDMA core.

5.3 Proposed Coyote Integration

Coyote separates trusted platform services from partially reconfigurable user regions, providing a natural location for bitstream inspection before PR [20]. We propose placing the checker in this trusted deployment path on a standalone, network-attached FPGA, as shown in Figure 3. A remote client submits a partial bitstream through Coyote's networking interface. Inspection and admission would execute on the FPGA without a local CPU or GPU.

Network ingress would retain the raw bitstream in HBM while forwarding its bytes to the checker. Preprocessing produces pixels incrementally for the quantized CNN, overlapping inspection with input reception. Retaining the original bytes is necessary because the downsampled CNN input cannot be used to configure the user region.

Trusted admission logic would compare the CNN's maliciousness score against the configured threshold. Only after acceptance and complete input reception would the PR controller read the staged bitstream and configure the selected user region. Rejected bitstreams would be discarded and logged.

The complete network-to-PR admission path remains proposed. This work implements and evaluates the FPGA preprocessing and CNN checker using host-submitted inputs, as described in Section 6.1. Connecting these components to network ingress, staging, and PR admission control remains integration work.

6 Evaluation

We evaluate the selected 256 × 256, seven-stage CNN and its implementation as an on-FPGA checker. Our latency comparison asks how quickly local inspection or GPU offloading can produce a verdict once the complete bitstream is available to the accelerator. We address four questions:

(1) **Detection quality.** How accurately does the checker distinguish benign partial bitstreams from bitstreams containing RO payloads?
(2) **Hardware feasibility.** What FPGA resources does the synthesized checker and its local-memory interface require?
(3) **Local inspection versus offloading.** How do the two paths compare in verdict latency, and what explains their differences?
(4) **Deployment overhead.** How significant is inspection latency relative to published invocation and partial-reconfiguration times?

6.1 Experimental Setup

We evaluate detection quality on the held-out test set defined in Section 4. The positive class is the malicious RO class. A false negative accepts a malicious bitstream, while a false positive rejects a benign accelerator. We use a probability threshold of 0.5 and report accuracy, F1, true-positive rate (TPR), false-negative rate (FNR), false-positive rate (FPR), and ROC AUC.

Benchmark platform and input availability. Figure 4 shows the measured setup. An AMD Alveo U55C provides local bitstream storage in HBM and executes the checker at 250 MHz. Its active PCIe link is Gen3 ×16; the AMD Instinct MI210 GPU uses Gen4 ×16. The FPGA and GPU attach to NUMA nodes 1 and 0, respectively. The figure shows logical data paths rather than a physical PCIe-switch topology.

The complete raw bitstream is in FPGA memory before each timed inspection. The host performs this preload and initiates execution, emulating deployment control that could reside on an embedded core. For the purposes of this study, we have a server with a CPU to initiate GPU execution, however in a commercial setting this could be done through the embedded CPU in a SoC FPGA, or by implementing a direct controller from the FPGA to CPU for starting execution as well.

Compared implementations. `FPGA_HLS4ML` performs local inspection using selective memory reads, sample-driven preprocessing, and our resource-optimized quantized CNN. `GPU_P2P` forwards the complete bitstream from FPGA memory to the GPU, executes the corresponding float model, and returns the verdict through FPGA-initiated mailbox reads. These are the OPT_10 and OPT_11 implementations in the benchmark artifact. Both are evaluated at batch size one.

Workloads and timing. The main comparison uses eight real bitstreams, R1–R8, ranging from 5.28 to 53.10 MiB. Each input and implementation has 20 warmup calls followed by 200 measured calls. We report medians and 95th percentiles (p95). The primary interval begins when the FPGA accepts the start command and ends when the classification verdict is available inside the FPGA.

This interval includes local memory access, preprocessing, inference, and, for GPU offloading, P2P transfer, runtime dispatch, synchronization, and verdict polling and retrieval. We exclude file access, GPU compilation, warmup and partial reconfiguration.

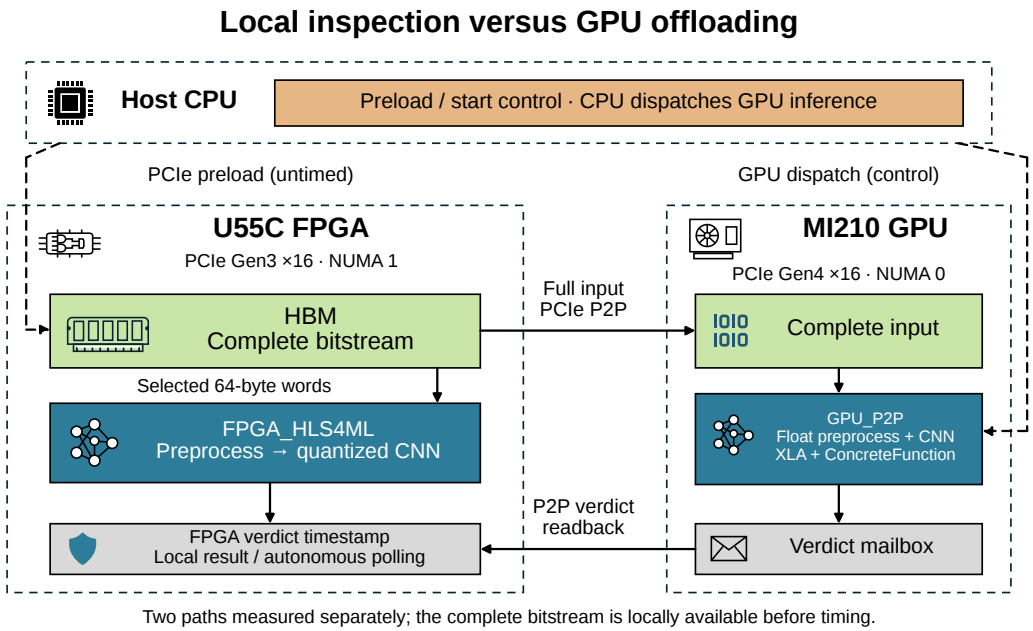


Fig. 4. Measured local-inspection and GPU-offload paths. Both start with the complete bitstream in FPGA memory. Local inspection reads selected words; offloading forwards the full input and retrieves a GPU-resident verdict through autonomous FPGA polling. Dashed paths show host preload and control. Preload is excluded from timing; GPU dispatch remains part of the measured execution path. The alternatives run separately.

Table 3. Detection quality of the selected 256^2-input, seven-layer checker on the held-out test set.

Accuracy	F1	TPR	FNR	FPR	ROC AUC
87.7%	88.5%	92.0%	8.0%	16.9%	95.1%

6.2 Detection Quality

Table 3 reports the detection quality of the selected checker on the held-out test set. The model reaches 87.7% accuracy and an F1 score of 88.5%. It detects 92.0% of the malicious bitstreams, corresponding to an FNR of 8.0%. Its ROC AUC of 95.1% indicates that the model provides substantial separation between the benign and malicious classes across possible decision thresholds.

The security-relevant error is the FNR because each false negative represents an malicious bitstream that would pass the checker. At the default threshold, the model misses 8.0% of malicious test samples. The corresponding FPR is 16.9%, meaning that the same operating point rejects a non-negligible fraction of benign applications.

This trade-off depends on the decision threshold. Reducing the threshold would reject more malicious bitstreams, but would also block more benign accelerators. Increasing it would reduce the operational cost of false positives while admitting more malicious designs. We use threshold 0.5 as a fixed and reproducible threshold throughout the evaluation. A production deployment could instead select the threshold according to an explicit platform policy, such as minimizing the FNR subject to a maximum acceptable FPR.

Table 4. Routed application-wrapper resources for local FPGA inspection (OPT_10), including memory access, interfaces, and instrumentation. The Coyote shell is excluded.

Resource	Count	U55C utilization
LUT	64,595	4.95%
FF	100,800	3.87%
BRAM	29 tiles	1.44%
DSP	664	7.36%
URAM	34 blocks	3.54%

6.3 Synthesized Checker and Optimizations

We synthesize the selected 256 × 256, seven-stage quantized model using hls4ml and place and route its wrapper for the U55C. The implementation combines our streaming optimization with FIFO and arithmetic-reuse tuning, leaving the trained model weights unchanged. A selective reader supplies the preprocessor with the memory words containing the sampled bytes.

Resource use. Table 4 reports the routed application-wrapper resources for local inspection. The wrapper uses 4.95% of device LUTs, 1.44% of BRAM, 7.36% of DSPs, and 3.54% of URAM. These counts include reader and interface logic and instrumentation; they are not CNN-only costs. The Coyote shell is excluded, and percentages use the complete U55C capacity.

Streaming and resource tuning. Our initial implementation buffered the complete preprocessed image before starting the CNN. We replaced this buffer with an HLS stream in a DATAFLOW region and launch the CNN once the first valid pixel is available. In the local-memory implementation, memory delivery, pixel production, and CNN input consumption overlap. The measurements below characterize this combined path; they do not isolate the latency benefit of streaming from other changes.

We reduce CNN memory and arithmetic costs through native hls4ml settings. FIFO candidates are derived using native FIFO optimization with a profiling depth of 1,024 entries and checked in board benchmarks. For DSPs, we increase reuse factors in later convolutional stages, setting `conv3`/`conv4` to 72 and `conv5`/`conv6` to 288. These changesconfigure hls4ml without custom modifications to its generated CNN kernels.

6.4 Local Inspection versus GPU Offloading

Both implementations start with the same raw bitstream available in FPGA memory and are measured until a verdict is available inside the FPGA. The comparison includes their different access paths and numerical precisions. It is not an isolated comparison of CNN compute latency.

GPU implementation. GPU_P2P uses the original floating-point model with batch size one. Its TensorFlow runtime combines preprocessing and inference in an XLA-compiled graph and invokes a cached ConcreteFunction for each input size. Once the full P2P transfer completes, the CPU dispatches the graph, synchronizes the result, and publishes it in a GPU mailbox. The FPGA polls this mailbox autonomously and timestamps the returned verdict.

Latency comparison. Figure 5 shows the eight real inputs. Initial-program FPGA median latency ranges from 2.679 to 3.074 ms, while GPU offloading ranges from 1.327 to 8.012 ms. Offloading is faster for our first bitstreams. Local inspection is faster for R3–R8, although its R3 advantage is only 1.05×. The local advantage reaches 2.91× at R8 (53.10 MiB).

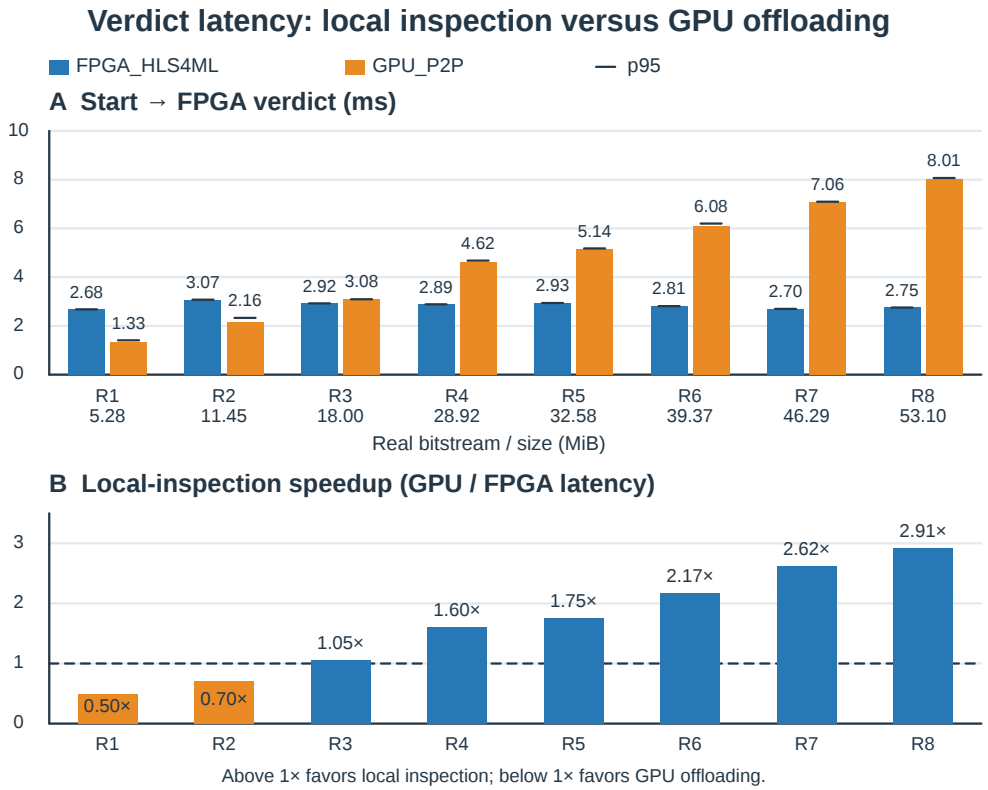


Fig. 5. Local inspection versus GPU offloading for real bitstreams already available to the FPGA. (A) Median start-to-verdict latency and p95 ticks across 200 calls after 20 warmups. Hollow markers show FPGA medians after two further identical-image programming cycles, with 200 and 20 measured calls. (B) GPU median divided by FPGA median; values above one favor local inspection. Bars use the initial FPGA state. Black ranges span the three programmed states, not confidence intervals. Both panels exclude input preload and PR.

Data movement and access patterns. The local reader requests only aligned 64-byte words containing sampled pixels. Each real input requires 65,536 distinct requests, totaling 4 MiB of requested data for 64 KiB of sampled bytes. GPU offloading instead forwards the complete raw bitstream, so its transfer cost grows with input size. Local inspection performs no PCIe input-payload transfer during checking.

Selective reads reduce traffic but incur substantial small-request overhead. Their measured requested-payload delivery rate is approximately 1.4–1.6 GB/s, whereas sequential forwarding reaches approximately 7.5 GB/s. These rates include gaps between returned words and do not measure physical HBM-wire traffic or peak memory bandwidth. The nearly constant selective-read volume helps explain the FPGA latency range, while efficient bulk transfer allows GPU offloading to win for smaller inputs. The comparison does not establish that FPGA arithmetic is intrinsically faster than GPU computation.

Execution breakdown. Figure 6 shows the R4 input (28.92 MiB), using one trial nearest each implementation's median. Local memory delivery, pixel production, and CNN input consumption

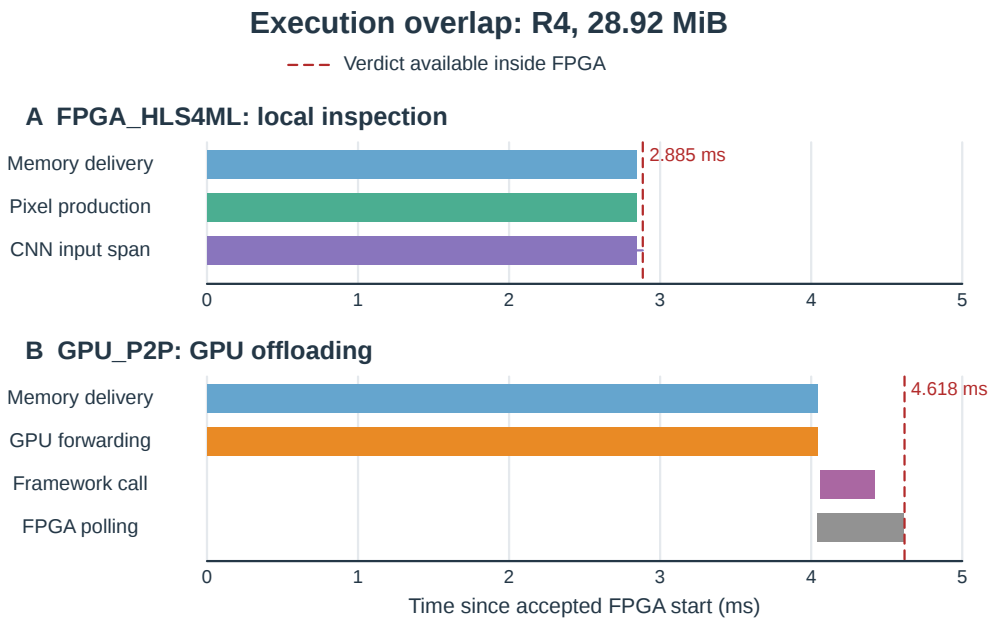


Fig. 6. Measured execution overlap for R4 (28.92 MiB), using the trials nearest each median on a shared time axis. Dashed lines mark verdict availability inside the FPGA. The CNN lane measures input consumption; its thin extension shows the output tail. GPU framework time includes dispatch and synchronization. Memory delivery, forwarding, and polling overlap other work and must not be summed as sequential phases.

overlap, with the verdict available at 2.885 ms. The CNN input span includes waiting for data; the short output tail completes after the final pixel is consumed.

GPU offloading forwards the input while reading it from FPGA memory. Forwarding ends at approximately 4.041 ms, followed by GPU execution and verdict publication. The FPGA obtains the verdict at 4.618 ms. The framework call includes dispatch, fused preprocessing/inference, and synchronization, rather than pure kernel execution. FPGA polling overlaps GPU work and publication. Its entire interval must not be added after those phases.

Repeatability limitation. Reprogramming the identical FPGA image changed some median latencies by up to 9.825%, exceeding the campaign's 5% cross-program stability threshold. A second run without reprogramming changed medians by at most 0.076%. An older transport adapter reproduced the second programmed state's result. The variation occurs in memory delivery, while exact outputs and the CNN output tail remain stable. Its physical cause is unresolved. Figure 5 therefore retains the initial measurements and shows the other programmed states rather than selecting only the fastest state. Narrow within-run p95 values do not imply stability across reprogramming.

6.5 Inspection Latency in Deployment Context

We compare the additional cost of inspection, once the bitstream is locally available, with published serverless invocation and PR times. The median size among our eight inputs is 30.75 MiB. We use the next measured size, R5 at 32.58 MiB (34,162,708 bytes), whose initial-program local-inspection median is 2.935 ms. Table 5 adds this interval to each reference, assuming inspection and the reference operation do not overlap.

Table 5. Contextual estimates adding 2.935 ms of local inspection to published reference times, assuming no overlap. Inspection starts with the complete bitstream available to the FPGA and excludes preload and network reception. These are not integrated-deployment measurements. The ICAP row is a throughput-based lower bound.

Reference	Baseline	With inspection	Increase
F3 warm invocation with reconfiguration [17]	141 ms	143.93 ms	2.08%
Coyote v2 shell PR #2 [20]	72.3 ms	75.23 ms	4.06%
Coyote v2 shell PR #3 [20]	85.5 ms	88.43 ms	3.43%
Coyote v2 ICAP at 32.58 MiB [20]	≈42.70 ms	≈45.64 ms	≈6.87%

F3 reports 141 ms for a warm SHA3 invocation including reconfiguration [17, Table 4]. Adding the selected inspection interval gives an estimated 2.08% increase. Coyote v2 reports kernel reconfiguration times of 72.3 ms for changing networking and numerical-kernel services (scenario #2), and 85.5 ms for removing a traffic sniffer while retaining RDMA (scenario #3) [20, Section 9.3, Table 3]. The corresponding increases are 4.06% and 3.43%. These kernel times exclude disk I/O and the host buffer copy and have a narrower scope than the F3 invocation.

Dividing 34,162,708 bytes by Coyote v2's reported UltraScale+ ICAP rate of 800 MB/s gives a size-matched lower bound of approximately 42.70 ms [20, Section 5.3, Table 2]. The selected inspection interval is 6.87% of that bound. This calculation excludes fixed PR overhead and does not establish sustained reconfiguration performance for this input.

These estimates use the initial programmed state and inherit the repeatability limitation in Section 6.4. They quantify inspection after input availability, not network acquisition, preload, or a complete PR admission path. The reference systems also have different timing scopes; the sums provide context rather than a measured integrated deployment.

7 TODO: Discussion + Conclusion

References

[1] Thea Aarrestad, Vladimir Loncar, Nicolò Ghielmetti, Maurizio Pierini, Sioni Summers, Jennifer Ngadiuba, Christoffer Petersson, Hampus Linander, Yutaro Iiyama, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Dylan Rankin, Sergo Jindariani, Kevin Pedro, Nhan Tran, Mia Liu, Edward Kreinar, Zhenbin Wu, and Duc Hoang. 2021. Fast convolutional neural networks on FPGAs with hls4ml. *Machine Learning: Science and Technology* 2, 4 (2021), 045015. doi:10.1088/2632-2153/ac0ea1

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, 265–283.

[3] Christophe Bobda, Joel Mandebi Mbongue, Paul Chow, Mohammad Ewais, Naif Tarafdar, Juan Camilo Vega, Ken Eguro, Dirk Koch, Suranga Handagala, Miriam Leeser, Martin Herbordt, Hafsah Shahzad, Peter Hofste, Burkhard Ringlein, Jakub Szefer, Ahmed Sanaullah, and Russell Tessier. 2022. The Future of FPGA Acceleration in Datacenters and the Cloud. *ACM Transactions on Reconfigurable Technology and Systems* 15, 3 (2022), 1–42. doi:10.1145/3506713

[4] Andrew Boutros, Mathew Hall, Nicolas Papernot, and Vaughn Betz. 2020. Neighbors From Hell: Voltage Attacks Against Deep Learning Accelerators on Multi-Tenant FPGAs. In *2020 International Conference on Field-Programmable Technology (ICFPT)*. 103–111. doi:10.1109/ICFPT51103.2020.00023

[5] Adrian M. Caulfield, Eric S. Chung, Andrew Putnam, Hari Angepat, Jeremy Fowers, Michael Haselman, Stephen Heil, Matt Humphrey, Puneet Kaur, Joo-Young Kim, Daniel Lo, Todd Massengill, Kalin Ovtcharov, Michael Papamichael, Lisa Woods, Sitaram Lanka, Derek Chiou, and Doug Burger. 2016. A Cloud-Scale Acceleration Architecture. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*. 1–13. doi:10.1109/MICRO.2016.7783710

[6] Jayeeta Chaudhuri and Krishnendu Chakrabarty. 2022. Detection of Malicious FPGA Bitstreams using CNN-Based Learning. In *2022 IEEE European Test Symposium (ETS)*. 1–2. doi:10.1109/ETS54262.2022.9810438

[7] Jayeeta Chaudhuri and Krishnendu Chakrabarty. 2023. Detection and Classification of Malicious Bitstreams for FPGAs. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. doi:10.1145/3566097.3568346

[8] Javier Duarte, Song Han, Philip Harris, Sergo Jindariani, Edward Kreinar, Benjamin Kreis, Jennifer Ngadiuba, Maurizio Pierini, Ryan Rivera, Nhan Tran, and Zhenbin Wu. 2018. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation* 13, 07 (2018), P07027. doi:10.1088/1748-0221/13/07/P07027

[9] Rana Elnaggar, Jayeeta Chaudhuri, Ramesh Karri, and Krishnendu Chakrabarty. 2023. Learning Malicious Circuits in FPGA Bitstreams. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 3 (2023), 726–739. doi:10.1109/TCAD.2022.3190771

[10] Dennis R. E. Gnad, Fabian Oboril, and Mehdi B. Tahoori. 2017. Voltage Drop-based Fault Attacks on FPGAs Using Valid Bitstreams. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. 1–7. doi:10.23919/FPL.2017.8056840

[11] Claudionor N. Coelho Jr., Aki Kuusela, Shan Li, Hao Zhuang, Jennifer Ngadiuba, Thea Klaeboe Aarrestad, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, and Sioni Summers. 2021. Automatic Heterogeneous Quantization of Deep Neural Networks for Low-Latency Inference on the Edge for Particle Detectors. *Nature Machine Intelligence* 3 (2021), 675–686. doi:10.1038/s42256-021-00356-5

[12] Ron Kohavi. 1995. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*. 1137–1145.

[13] Dario Korolija, Timothy Roscoe, and Gustavo Alonso. 2020. Do OS Abstractions Make Sense on FPGAs?. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 991–1010.

[14] Tuan La, Khoa Dang Pham, Joseph Powell, and Dirk Koch. 2021. Denial-of-Service on FPGA-based Cloud Infrastructures: Attack and Defense. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 3 (2021), 441–464. doi:10.46586/tches.v2021.i3.441-464

[15] Tuan Minh La, Kaspar Mätas, Nikola Grunchevski, Khoa Dang Pham, and Dirk Koch. 2020. FPGADefender: Malicious Self-Oscillator Scanning for Xilinx UltraScale+ FPGAs. *ACM Transactions on Reconfigurable Technology and Systems* 13, 3 (2020), 15:1–15:31. doi:10.1145/3402937

[16] Dina G Mahmoud, Ognjen Glamočanin, Francesco Regazzoni, and Mirjana Stojilović. 2023. Practical Implementations of Remote Power Side-Channel and Fault-Injection Attacks on Multitenant FPGAs. In *Security of FPGA-Accelerated Cloud Computing Environments*. Springer, 101–135.

[17] Charalampos Mainas, Martin Lambeck, Bruno Scheufler, Laurent Bindschaedler, Atsushi Koshiba, and Pramod Bhatotia. 2025. F3: An FPGA-accelerated FaaS Framework. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. doi:10.1145/3731545.3731582

[18] Alessandro Palumbo, Luca Cassano, Biagio Luzzi, Juan Antonio Hernández, Pedro Reviriego, Giuseppe Bianchi, and Marco Ottavi. 2022. Is your FPGA bitstream Hardware Trojan-free? Machine learning can provide an answer. *Journal of Systems Architecture* 128 (2022), 102543. doi:10.1016/j.sysarc.2022.102543

[19] George Provelengios, Daniel Holcomb, and Russell Tessier. 2020. Power Wasting Circuits for Cloud FPGA Attacks. In *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*.

[20] Benjamin Ramhorst, Dario Korolija, Maximilian Jakob Heer, Jonas Dann, Luhao Liu, and Gustavo Alonso. 2025. Coyote v2: Raising the Level of Abstraction for Data Center FPGAs. In *Proceedings of the ACM Symposium on Operating Systems Principles*. arXiv:2504.21538 [cs.AR] doi:10.1145/3731569.3764845

[21] Jens Rettkowski, Safdar Mahmood, Arij Shallufa, Michael Hübner, and Diana Göhringer. 2019. Inspection of Partial Bitstreams for FPGAs Using Artificial Neural Networks. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 83–86. doi:10.1109/IPDPSW.2019.00023

[22] Rye Stahle-Smith, Carter Antley, Jason D. Bakos, and Rasha Karakchi. 2026. Hardware-Accelerated Line-Rate Bitstream Screening for Secure FPGA Reconfiguration. arXiv:2605.08984 [cs.CR] https://arxiv.org/abs/2605.08984

[23] Rye Stahle-Smith and Rasha Karakchi. 2025. Real-time ML-based Defense Against Malicious Payload in Reconfigurable Embedded Systems. arXiv:2509.02387 [cs.CR] https://arxiv.org/abs/2509.02387

[24] Junghwan Yoon, Yeseul Seo, Jihun Jang, Minsoo Cho, Jaeseung Kim, Hyung-Min Kim, and Taesoo Kwon. 2018. A Bitstream Reverse Engineering Tool for FPGA Hardware Trojan Detection. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 2318–2320. doi:10.1145/3243734.3278487

Received TODO; revised TODO; accepted TODO